FORMATIVE 1

Metro Interstate Traffic Volume Forecasting Pipeline

A complete time series pipeline covering preprocessing, exploratory analysis, forecasting, MySQL, MongoDB, CRUD APIs and integrated prediction.

Dataset	Time range	Target
Metro Interstate Traffic Volume	2 Oct 2012 to 30 Sep 2018	Hourly traffic volume

Course: Machine Learning Techniques
Group: [Add group name]
GitHub: https://github.com/irachrist1/formative-1-time-series-pipeline

Technical summary

The project predicts hourly westbound traffic on Interstate 94 using weather, calendar, lag and moving average features. The selected RF 2 random forest achieved a test MAE of **147.79**, RMSE of **233.21** and R squared of **0.9861** on the final chronological test period.

The strongest evidence is the repeated weekly traffic pattern. Traffic one week earlier has a correlation of 0.944 with current traffic. A normalized four table MySQL schema and a nested MongoDB collection store the same hourly records. FastAPI exposes complete CRUD operations plus latest and date range queries for both databases.

The end to end client fetched 337 records from the running API, recreated the training features, loaded the saved model and predicted 1,110.93 vehicles for the latest record. The actual volume was 954, giving an absolute error of 156.93 vehicles.

Project scope

The findings describe one westbound I 94 monitoring station between Minneapolis and St. Paul, Minnesota. The model should be retrained before use on another road or country because traffic schedules and road conditions differ.

Repository outputs

Component	Evidence
Task 1	Executed notebook, seven analytical figures, cleaned CSV and model experiment table
Task 2	MySQL schema, ERD, MongoDB design, sample documents, queries and results
Task 3	FastAPI application and integration tests for SQL and MongoDB
Task 4	API prediction client and saved forecast result

Task 1: Dataset understanding and preprocessing

Problem definition and justification

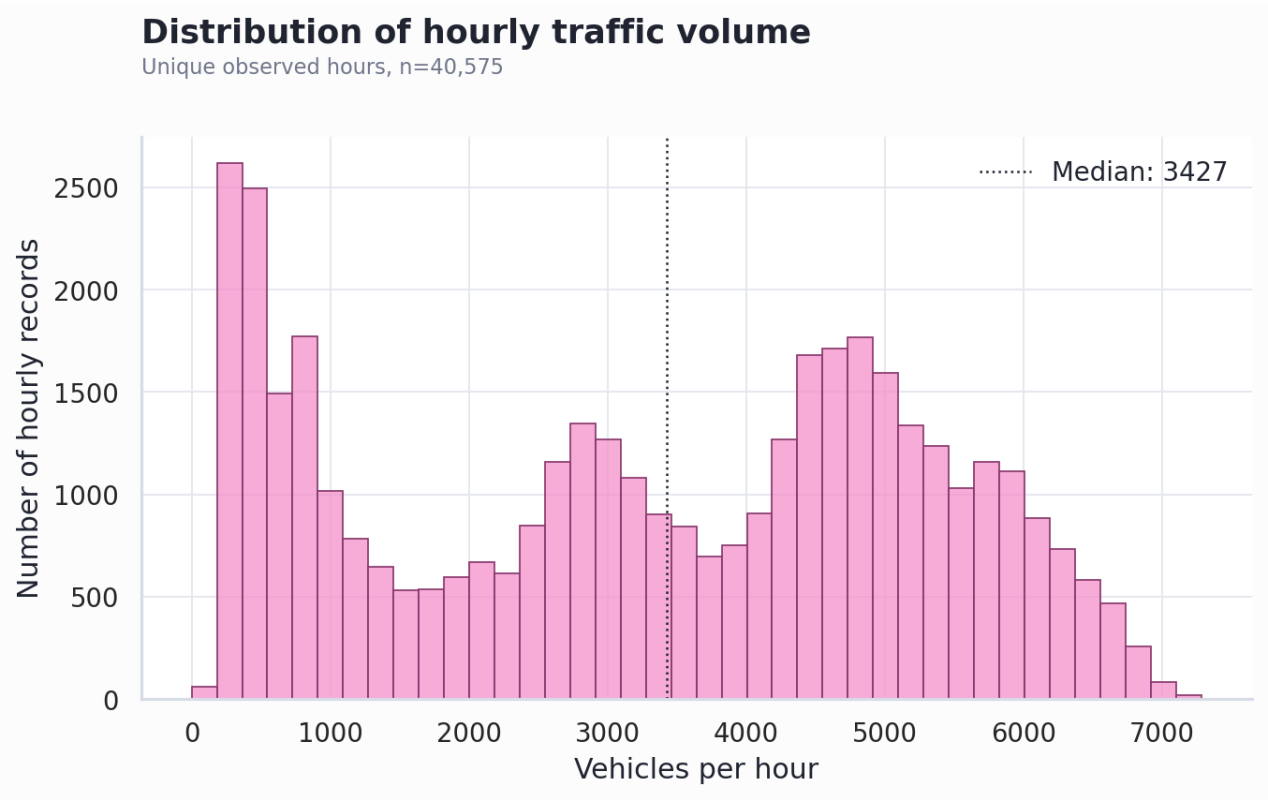
The forecasting target is **traffic_volume**, the number of westbound vehicles recorded during an hour. Forecasts can help traffic managers understand expected congestion and plan road operations. This dataset is suitable because it has a clear timestamp, a continuous target, weather measurements, holiday labels and more than six years of observations.

Item	Observed value
Raw shape	48,204 rows and 9 columns
Time range	2 October 2012 09:00 to 30 September 2018 23:00
Intended frequency	Hourly
Unique observed hours	40,575
Repeated timestamp rows	7,629
Fully duplicated rows	17
Absent hours after regular indexing	11,976

Missing data decisions

The holiday label `None` means a normal day, not an unknown value, so it was renamed `No Holiday`. Fully duplicated rows were removed. Repeated timestamps represent multiple weather descriptions during the same hour, so numeric values were averaged and categorical weather descriptions were combined.

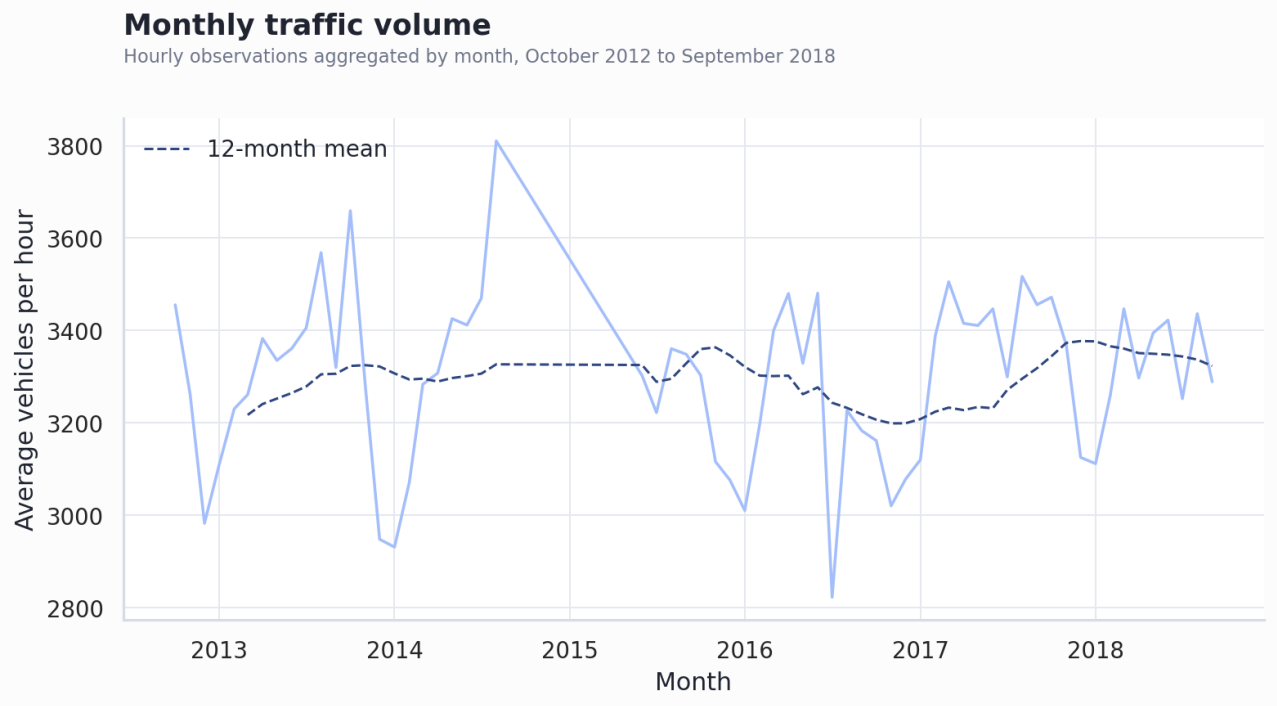
The data was reindexed to an hourly timeline. Internal gaps of at most three hours were interpolated using time order. Longer gaps remained missing and model rows that depended on those gaps were removed. This approach uses nearby information for small interruptions without fabricating long traffic sequences.



Hourly volume ranges from zero to 7,280 vehicles. The mean is 3,260, the median is 3,380, and the standard deviation is 1,987. The broad shape reflects overnight lows and commuting peaks.

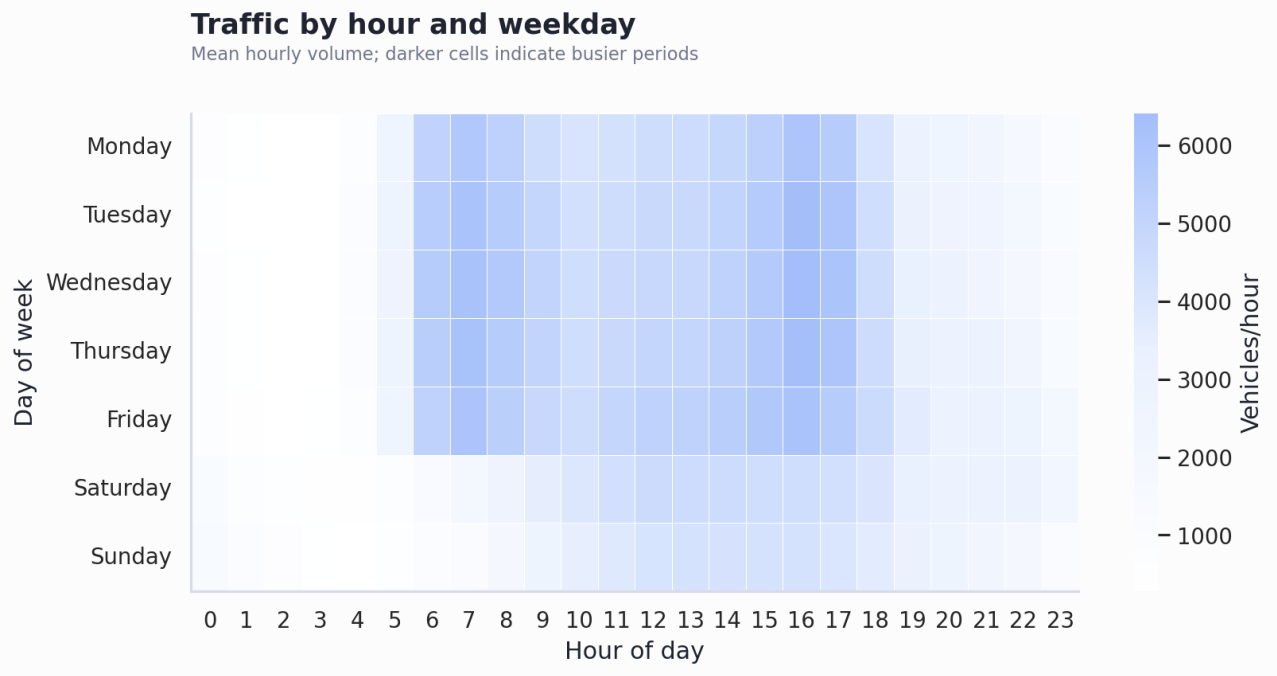
Six analytical questions

1. Does traffic have a long term increasing or decreasing trend?



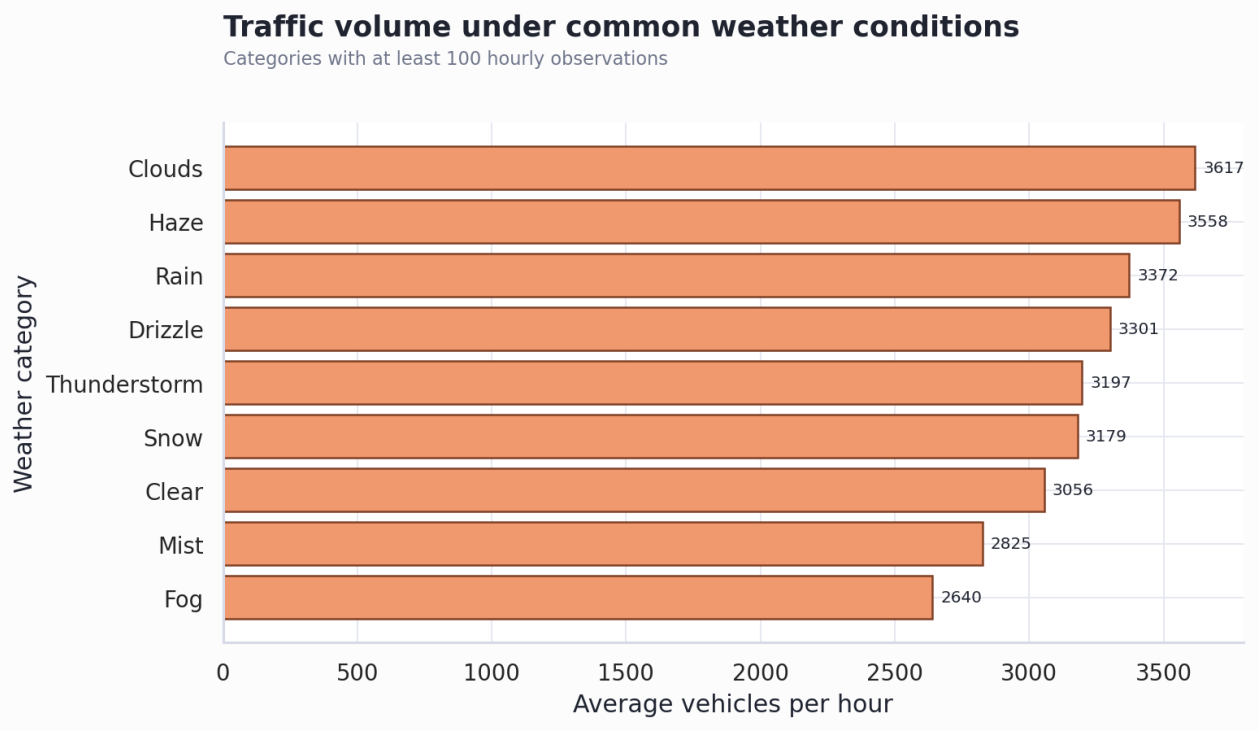
The first twelve months averaged 3,306 vehicles per hour and the last twelve averaged 3,333. The difference is 0.8 percent. There is no strong monotonic trend; seasonal movement and data coverage create larger month to month changes.

2. Which weekday and hour combination has the highest traffic?



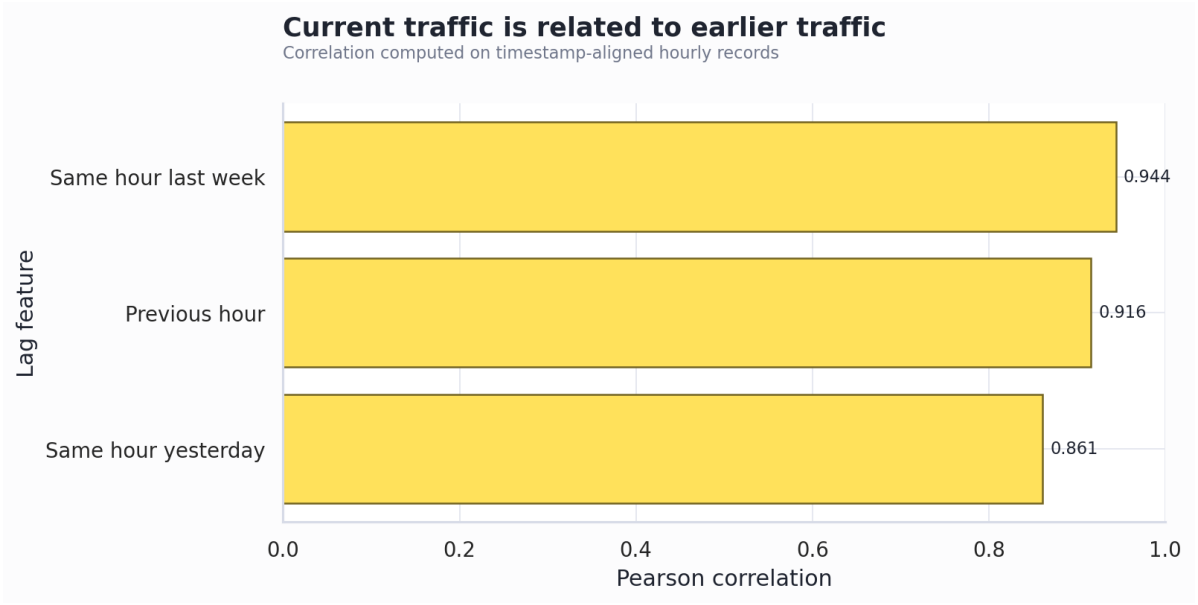
The highest mean occurs on Wednesday at 16:00, with about 6,414 vehicles per hour. Weekday commuter hours are consistently busier than overnight and weekend periods, supporting hour and weekday model features.

3. Does traffic volume differ across weather conditions?



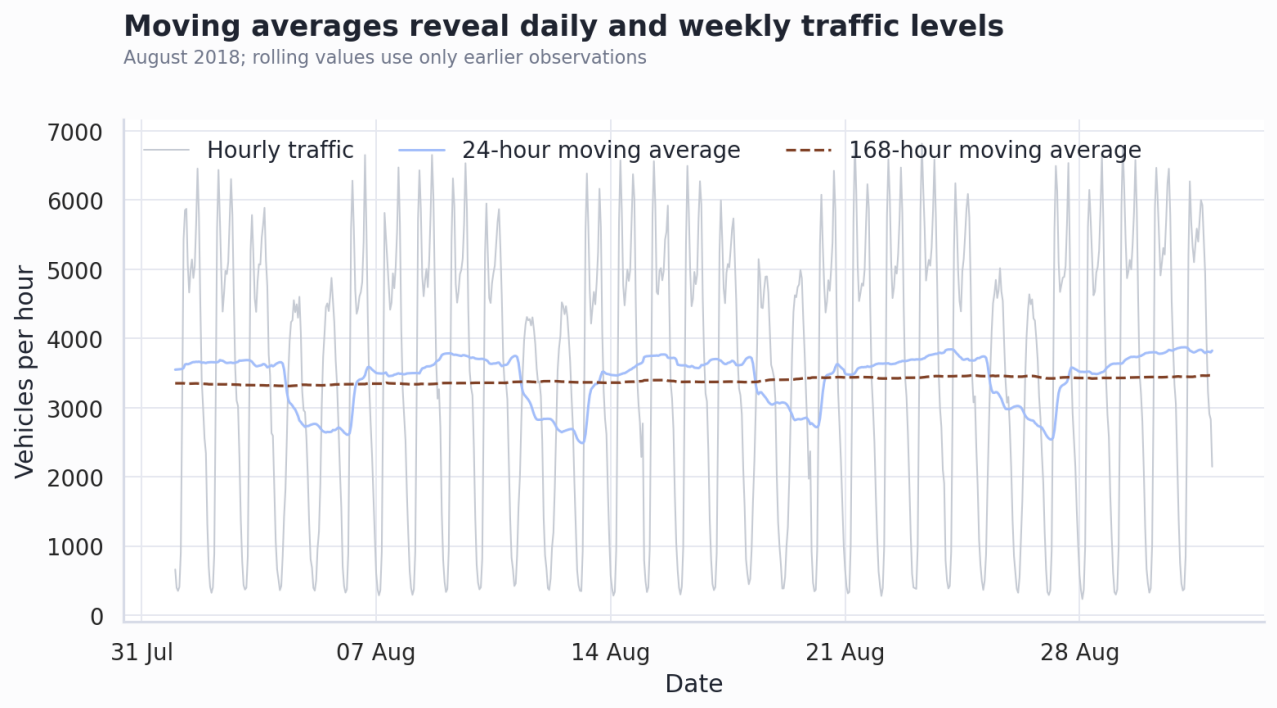
Among weather categories with at least 100 records, Clouds has the highest mean at 3,617, while Fog has the lowest at 2,640. This result is descriptive because weather categories are also related to season and time of day.

4. Is current traffic related to traffic 1, 24 and 168 hours earlier?



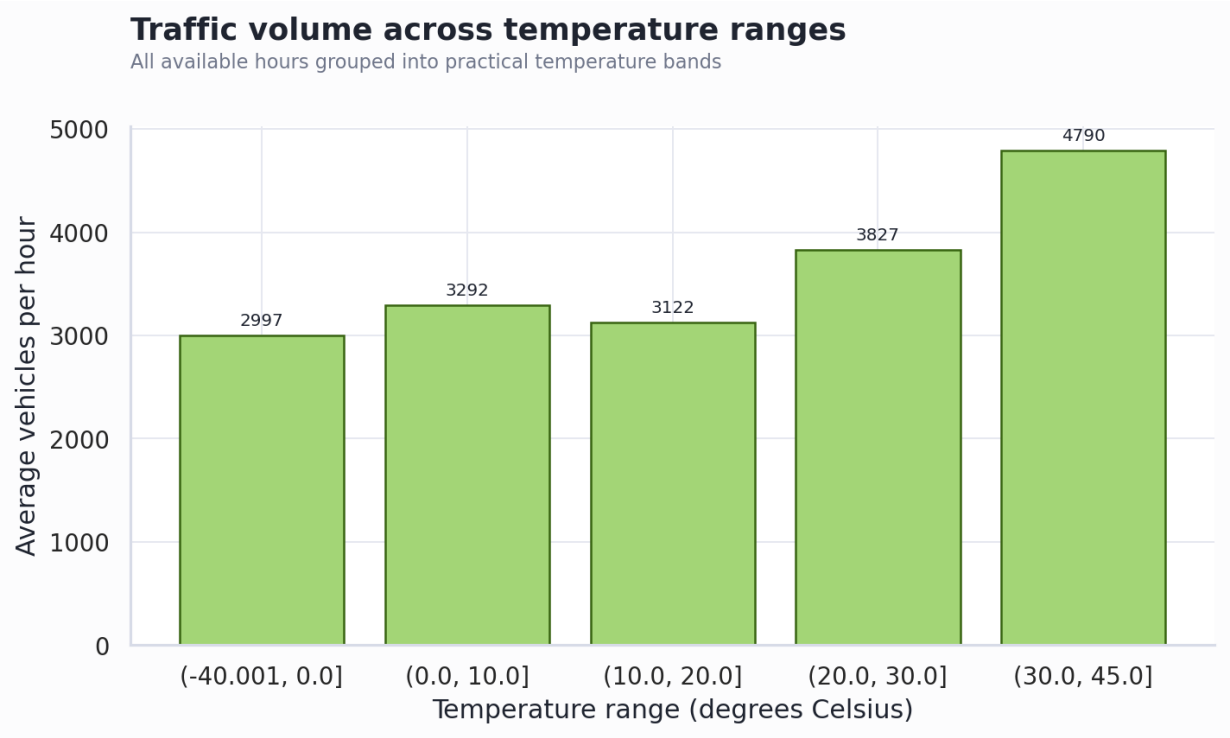
The 1 hour lag correlation is 0.916, the 24 hour lag is 0.861, and the 168 hour lag is 0.944. The weekly lag is the strongest, showing that travel schedules repeat across weeks.

5. What do 24 hour and 168 hour moving averages reveal?



During August 2018 the hourly standard deviation was 1,956. The 24 hour moving average standard deviation was 370, while the 168 hour average was only 45. The weekly moving average gives a stable baseline and the daily average reacts faster. Both use only earlier traffic values.

6. How does traffic differ by temperature range?



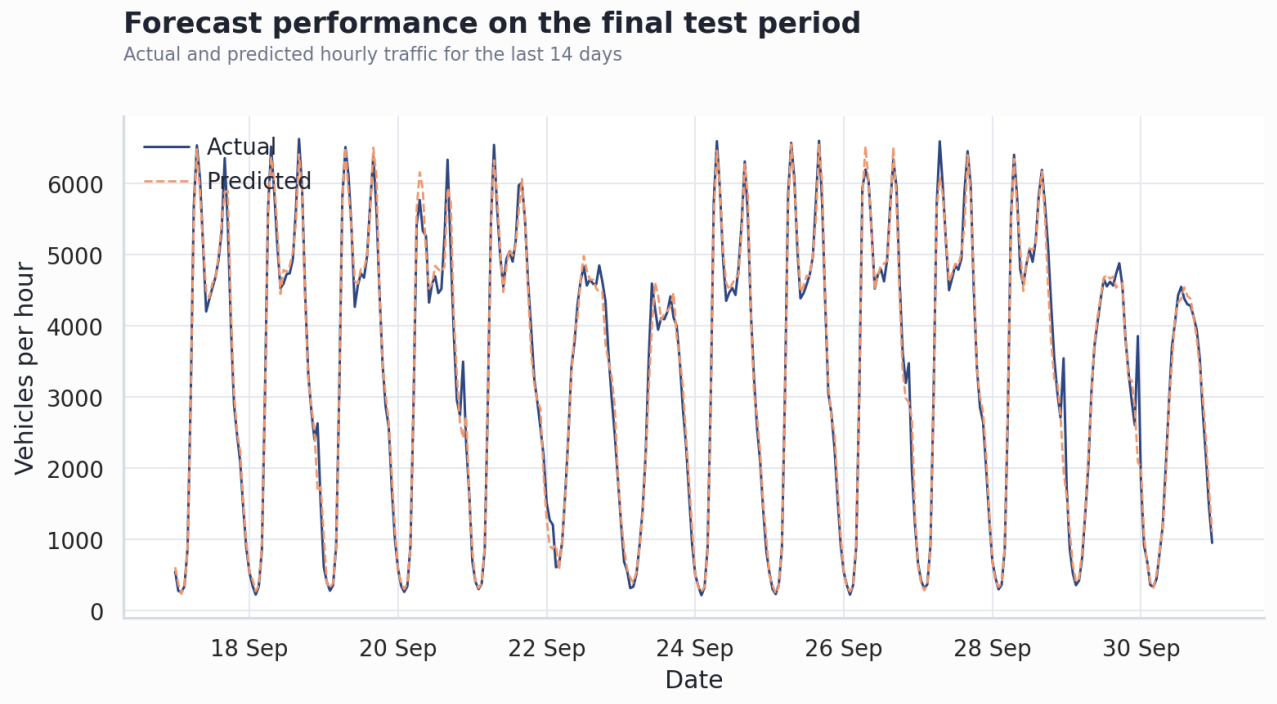
Traffic is not a simple linear function of temperature. Warm bands show higher averages, but the band above 30 degrees Celsius contains only 396 observations. Time of day and season can explain part of this difference, so temperature should be used with calendar and lag variables.

Model experiments and validation

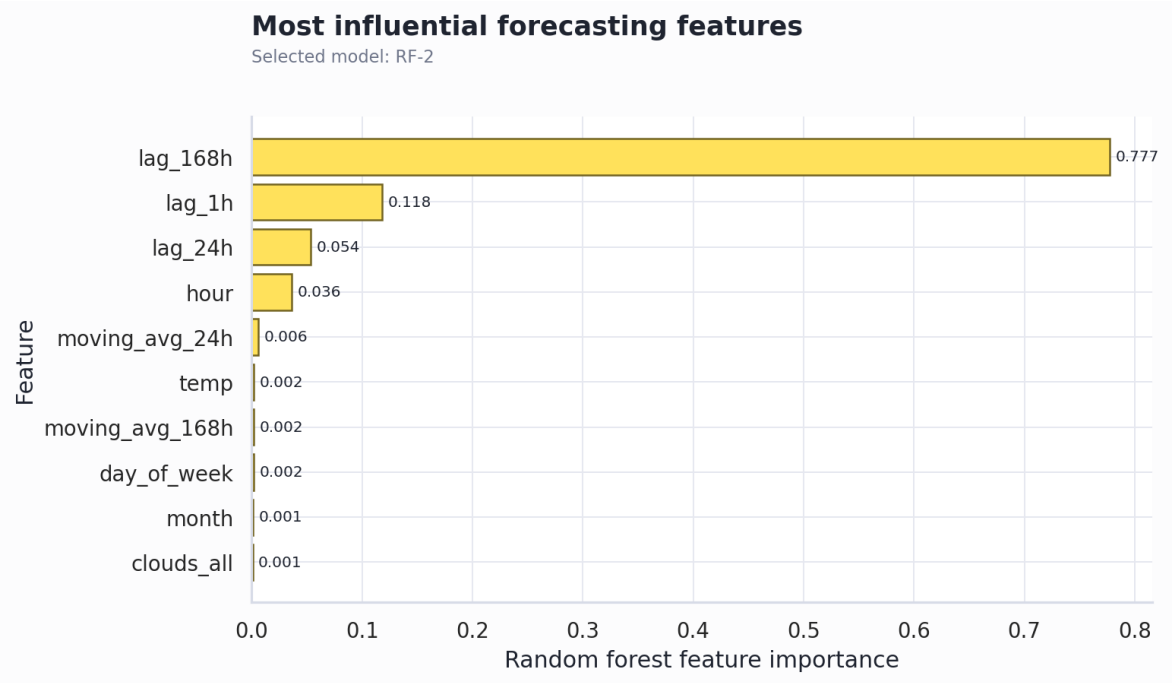
Features include temperature, rain, snow, clouds, hour, weekday, month, weekend and holiday flags, traffic lags at 1, 24 and 168 hours, and past only moving averages over 24 and 168 hours. The chronological split used 70 percent for training, 15 percent for validation and 15 percent for testing.

Experiment	Trees	Depth	Min leaf	Val MAE	Val RMSE	Val R2
RF-2	100	18	2	158.86	246.30	0.9845
RF-3	140	None	2	158.92	246.61	0.9844
RF-1	60	12	4	163.94	253.71	0.9835

RF 2 was selected using the lowest validation RMSE. After refitting on training plus validation data, it achieved test MAE 147.79, RMSE 233.21 and R squared 0.9861 across 6,281 test rows.



Predictions closely follow daily peaks and overnight lows during the last fourteen days. Remaining errors occur around abrupt deviations and unusual days.

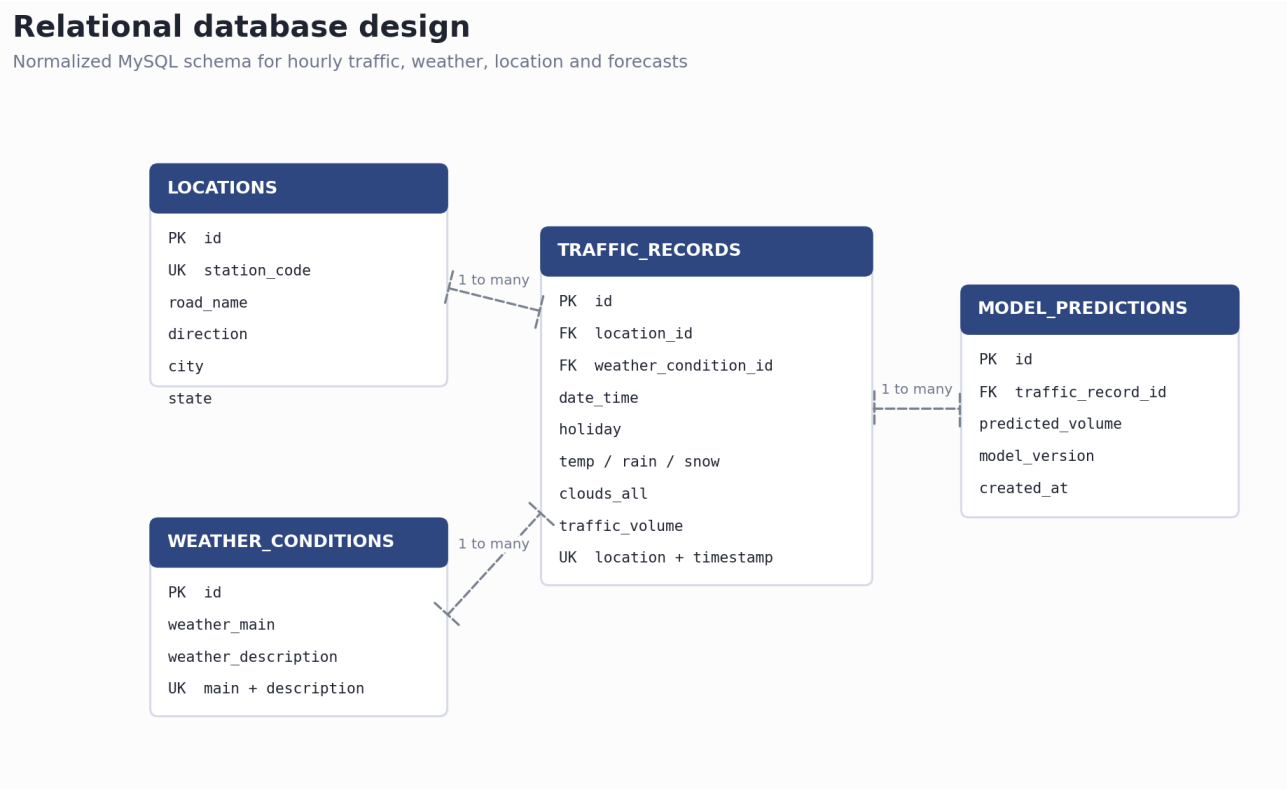


Lag and calendar variables are the most important. This is consistent with the EDA and shows that repeated schedules are more predictive than weather alone.

Task 2: Database design and implementation

MySQL relational design

The normalized schema has four tables. Locations and weather conditions store reusable descriptions, traffic records store time varying measurements, and model predictions retain forecast history. Unique and time indexes support integrity and efficient chronological retrieval.



MongoDB collection design

Each MongoDB document represents one hourly observation. Weather fields are embedded because they are normally read together. The collection uses a unique date_time index and a compound location_id plus descending date_time index. This supports latest record and range access without joins.

Design	MySQL	MongoDB
Main unit	traffic_records row	traffic_records document
Weather	Foreign key to weather_conditions	Embedded weather object
Location	Foreign key to locations	location_id field
Time index	date_time and location/date_time	date_time and location/date_time
Prediction history	model_predictions table	Can be a separate collection

Database queries and executed results

MySQL query examples

```
-- Latest record
SELECT tr.date_time, tr.traffic_volume, wc.weather_main
FROM traffic_records tr
JOIN weather_conditions wc ON wc.id = tr.weather_condition_id
ORDER BY tr.date_time DESC LIMIT 1;

-- Records by date range
SELECT date_time, traffic_volume FROM traffic_records
WHERE date_time BETWEEN '2018-09-01' AND '2018-09-07 23:59:59';

-- Average by weather
SELECT wc.weather_main, AVG(tr.traffic_volume), COUNT(*)
FROM traffic_records tr JOIN weather_conditions wc ON wc.id=tr.weather_condition_id
GROUP BY wc.weather_main ORDER BY AVG(tr.traffic_volume) DESC;
```

MongoDB query examples

```
// Latest record
db.traffic_records.find({}).sort({date_time: -1}).limit(1)

// Records by date range
db.traffic_records.find({date_time: {$gte: ISODate('2018-09-01'),
  $lte: ISODate('2018-09-07T23:59:59Z')}}).sort({date_time: 1})

// Average by weather
db.traffic_records.aggregate([
  {$group: {_id: '$weather.weather_main',
    average_traffic: {$avg: '$traffic_volume'}, records: {$sum: 1}}},
  {$sort: {average_traffic: -1}}
])
```

Database	Query	Executed result
MySQL	Latest	30 Sep 2018 23:00, 954 vehicles, Clouds
MySQL	Date range	168 rows from 1 to 7 September
MySQL	Weather average	Clouds first: 3,929.22
MongoDB	Latest	30 Sep 2018 23:00, 954 vehicles, Clouds
MongoDB	Date range	168 documents from 1 to 7 September
MongoDB	Weather average	Clouds first: 3,929.22

Both services were installed locally and loaded with the same 1,000 recent hourly records. Matching results confirm that the two implementations represent the same data.

Task 3: CRUD and time series endpoints

FastAPI provides matching operations for both storage systems. Input validation rejects unrealistic temperature, negative precipitation, invalid cloud percentages and negative traffic. Duplicate timestamps return HTTP 409 and missing records return HTTP 404.

Method	SQL endpoint	MongoDB endpoint	Purpose
POST	/sql/records	/mongo/records	Create
GET	/sql/records/{id}	/mongo/records/{id}	Read one
PUT	/sql/records/{id}	/mongo/records/{id}	Update
DELETE	/sql/records/{id}	/mongo/records/{id}	Delete
GET	/sql/records/latest	/mongo/records/latest	Latest
GET	/sql/records?start=&end=	/mongo/records?start=&end=	Date range

The automated integration test performs create, read, date range, latest, update and delete operations for SQL and MongoDB. The suite passes. Interactive OpenAPI documentation is available at `http://127.0.0.1:8000/docs` when the server runs.

Example create request

```
POST /sql/records
{
  "date_time": "2018-10-01T00:00:00",
  "temp": 282.5,
  "rain_1h": 0.0,
  "snow_1h": 0.0,
  "clouds_all": 75,
  "weather_main": "Clouds",
  "weather_description": "broken clouds",
  "traffic_volume": 1000,
  "location_id": 1
}
```

Task 4: Integrated prediction script

The prediction script requests the latest record and the previous fourteen days from either database route. It parses the timestamps, applies the same shifted lag and moving average logic used during training, loads the saved RF 2 artifact, and predicts the latest traffic volume.

Field	Observed result
Database route	sql
API history records	337
Record timestamp	2018-09-30 23:00:00
Actual traffic	954
Predicted traffic	1110.93
Absolute error	156.93
Model	RF-2

```
# Start the API
uvicorn src.api.main:app --reload

# Predict through either database
python scripts/predict_from_api.py --database sql
python scripts/predict_from_api.py --database mongo
```

Both database routes returned the same feature history and produced the same forecast. This verifies the required path from an API record through preprocessing and model loading to a final prediction.

Limitations and robustness

The random forest estimates expected traffic from historical patterns but does not establish causal weather effects. Missing periods reduce the usable training set. A single chronological holdout is appropriate for the assignment, while a production system should add rolling origin cross validation, a seasonal naive baseline, drift monitoring and retraining on newer local data.

Team participation and reproducibility

Member	Student ID	Major contribution
[Member 1 name]	[ID]	Data preprocessing, exploratory analysis and visualizations
[Member 2 name]	[ID]	Feature engineering, model experiments and evaluation
[Member 3 name]	[ID]	MySQL and MongoDB design, implementation and queries
[Member 4 name]	[ID]	FastAPI CRUD endpoints, integration testing and prediction script

GitHub repository: https://github.com/irachrist1/formative-1-time-series-pipeline

Each member should have at least four clear commits under their own GitHub account. Commit messages should identify meaningful work such as preprocessing, model experiments, database schema, API routes, tests or report evidence.

Reproduction

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python scripts/run_analysis.py
python scripts/seed_databases.py
uvicorn src.api.main:app --reload
pytest -q
```

Conclusion

The project implements the full requested pipeline in one structured repository. It includes documented preprocessing, six analytical questions, lag and moving average features, three tuned experiments, two database systems, complete CRUD endpoints, time series queries and an end to end API prediction.

Dataset reference

Metro Interstate Traffic Volume, Kaggle. Original measurements from the Minnesota Department of Transportation and distributed through the UCI Machine Learning Repository.
https://www.kaggle.com/datasets/anshtanwar/metro-interstate-traffic-volume